Resume Analyzer

Viva Questions and Model Answers

A project-specific preparation guide based on the repository README, application implementation, and knowledge-base rubric.

Use these answers as a starting point. Personalize contribution-related answers and verify details against your own experience.

Project overview

Q1. What is the Resume Analyzer project?

Suggested answer: It is a Generative AI application that compares an uploaded resume with role-specific skill benchmarks and provides feedback on missing skills, relevant experience, and ways to improve resume bullets.

Q2. What problem does the project solve?

Suggested answer: Students may not know how their resumes align with a target technical role. The application helps surface relevant evidence and gaps, and suggests more specific, measurable wording for existing resume bullets.

Q3. Who are the intended users?

Suggested answer: Students and job seekers who want feedback on a resume for one of the technical roles supported by the application.

Q4. Which target roles are supported?

Suggested answer: Cloud and DevSecOps Engineer, Software Engineer, Data Analyst, AI/ML Engineer, Cybersecurity Engineer, and Game Developer.

Q5. What are the main outputs?

Suggested answer: The interface presents three sections: Missing Skills, Relevant Experience, and Areas to Improve. The last section can include before-and-after bullet rewrites and ATS-related observations.

Q6. What technologies are used?

Suggested answer: The project is written in Python. It uses Gradio for the UI; LangChain for prompt construction, text splitting, document loading, and pipeline composition; Chroma for vector search; Hugging Face sentence-transformers for local embeddings; and a chat model accessed through an OpenAI-compatible API.

Architecture and processing

Q7. Walk through the end-to-end analysis flow.

Suggested answer: The user uploads a PDF or DOCX and selects a role. The app extracts and cleans resume text, splits and embeds it into a temporary Chroma collection, retrieves role and rubric context from the knowledge-base index, retrieves relevant resume passages, sends those contexts to a prompt-driven chat model, then displays the report in three UI panels.

Q8. What is retrieval-augmented generation (RAG)?

Suggested answer: RAG retrieves relevant external text and supplies it as context to a language model when generating an answer. Here, the retrieved context comes from role benchmarks, the resume, and the improvement rubric, helping tailor the report to both the chosen role and the actual resume.

Q9. What documents make up the knowledge base?

Suggested answer: Text and Markdown files in the knowledge_base directory. They contain role benchmarks and a resume-improvement rubric. The app reads supported files, assigns metadata, splits their text into chunks, and indexes the chunks in Chroma.

Q10. How is the knowledge base initialized?

Suggested answer: At startup, the app ensures the knowledge_base directory and default files exist, then builds a Chroma vector store from its text and Markdown documents. The in-memory knowledge-base store is reused for subsequent analyses in that running process.

Q11. How is an uploaded resume processed?

Suggested answer: The app selects a PDF or DOCX document loader based on the file extension, extracts text, normalizes whitespace, rejects a document if no text can be extracted, splits the content into chunks, adds a student_resume metadata tag, and creates a temporary Chroma vector store.

Q12. What are the chunk sizes and why use overlap?

Suggested answer: Knowledge-base text uses 500-character chunks with 80 characters of overlap; resume text uses 600-character chunks with 100 characters of overlap. Chunking keeps retrieval focused, and overlap helps retain context that would otherwise fall across chunk boundaries.

Q13. What is the purpose of embeddings?

Suggested answer: An embedding maps text to a numeric vector so semantically related text can be found by similarity rather than exact keyword matching. The app uses sentence-transformers/all-MiniLM-L6-v2 locally to embed both indexed documents and search queries.

Q14. How does the retrieval stage work?

Suggested answer: First, it retrieves up to six chunks for the selected role using role metadata, and up to four rubric chunks using rubric metadata. It then uses the role benchmark text as a semantic query to retrieve up to eight relevant resume chunks. The resulting three contexts are passed to the generation step.

Q15. Why use metadata filters in Chroma?

Suggested answer: Metadata filters constrain search to the intended document category or role. This helps avoid mixing another role's benchmarks into the analysis and separates rubric content from role benchmarks and resume content.

Q16. What does the prompt ask the language model to do?

Suggested answer: It asks for exactly three Markdown sections: missing skills, relevant experience, and areas to improve. It instructs the model to use only the supplied context, not invent resume experience, and to show before-and-after rewrites using the XYZ formula.

Q17. What is the XYZ formula?

Suggested answer: It is a resume bullet structure: 'Accomplished X as measured by Y by doing Z.' X is the outcome, Y is the measurable impact, and Z is the approach or technology used. The rubric uses it to encourage concrete, evidence-based bullets.

Q18. What does LCEL do in this application?

Suggested answer: LangChain Expression Language composes the prompt, chat model, and string output parser into a single chain. The application invokes that chain with the selected role and the three retrieved contexts.

Design choices and implementation

Q19. Why use both a knowledge base and the uploaded resume?

Suggested answer: The knowledge base describes what a role expects and how to improve a resume. The uploaded resume provides evidence about the individual. Comparing both lets the system identify gaps and matches instead of generating generic career advice.

Q20. Why use a local embedding model with a hosted chat model?

Suggested answer: The code uses a local Hugging Face embedding model because the configured chat API is treated as chat-only and does not provide the required embeddings endpoint. The chat model is used for natural-language synthesis; embeddings support semantic retrieval locally.

Q21. How is the chat model configured?

Suggested answer: The application reads NEXUS_API_KEY from environment configuration, and allows BASE_URL and MODEL_NAME to be overridden. It constructs a ChatOpenAI client with a low temperature of 0.2 and the configured OpenAI-compatible endpoint.

Q22. Why is a low temperature used?

Suggested answer: A lower temperature generally makes responses less variable. For structured resume feedback, consistency and adherence to provided evidence are more useful than highly creative output. It does not guarantee factual correctness.

Q23. What role does Gradio play?

Suggested answer: Gradio provides the browser-based interface, including file upload, target-role selection, Analyze and Reset buttons, a status message, and three Markdown output cards.

Q24. How is the model report divided for display?

Suggested answer: The app looks for the expected Markdown headings and extracts each section into a separate output panel. If headings are not found, it has fallback behavior to display the report rather than silently discard all generated text.

Q25. What happens if no resume or role is supplied?

Suggested answer: The analysis handler returns a user-facing prompt to upload a supported resume or select a target role before proceeding.

Q26. Why create a temporary resume vector store?

Suggested answer: The resume is indexed for retrieval during analysis, while the knowledge base is shared and reused. A temporary per-upload collection avoids treating one user's resume as part of the shared role benchmarks.

Quality, limitations, and responsible use

Q27. How does the project reduce hallucinations?

Suggested answer: The prompt tells the model to ground its response only in the retrieved role benchmark, resume passages, and rubric, and to avoid inventing skills or experience. Retrieval provides relevant evidence. These are mitigations, not a guarantee, so generated feedback should be reviewed.

Q28. Does the application support scanned or image-only PDFs?

Suggested answer: No. The project extracts a text layer from PDFs and does not implement OCR. If no text is extracted, it reports that the resume may be scanned or image-only and asks for a text-based document.

Q29. What are the privacy considerations?

Suggested answer: Resume text is sent to the configured chat API to generate feedback, while the embedding model runs locally. Users should understand this data flow, and API credentials should be kept out of source control. The README advises keeping the .env file private.

Q30. Is the knowledge-base vector store persistent?

Suggested answer: The current Chroma configuration creates the knowledge-base collection in memory; the persistence directory is commented out. The index is rebuilt when the application starts, so it is not retained across restarts.

Q31. What are the main limitations?

Suggested answer: The system depends on text extraction quality, the completeness of its role benchmarks, semantic retrieval quality, and the chat model's output. It does not perform OCR, and generated suggestions need human review. The current design is a local Gradio application rather than a production multi-user service.

Q32. How would you evaluate the quality of the results?

Suggested answer: Use a representative set of resumes and role targets, then have qualified reviewers assess whether retrieved passages are relevant, missing-skill claims are supported, experience matches are accurate, and rewrites preserve the original facts. Track both quality and operational measures such as latency and failure rate.

Q33. How would you test the application?

Suggested answer: Add unit tests for text cleaning, supported file handling, metadata tagging, role selection, retrieval filters, report-section parsing, and validation messages. Add integration tests for document ingestion and the retrieval-to-generation flow, using mocked model calls where appropriate.

Q34. How can you prevent the model from inventing resume metrics?

Suggested answer: Require every suggested rewrite to use only quantities present in the resume, or clearly mark missing measurements as placeholders for the user to verify. A production version should validate generated output and require human approval before reuse.

Future work and defense questions

Q35. What improvements would you prioritize next?

Suggested answer: I would first add automated tests and a curated evaluation set, then improve extraction and feedback validation. Depending on measured needs, later work could add OCR for scanned resumes, persistent and versioned indexes, stronger privacy controls, and usage monitoring.

Q36. How could you improve retrieval quality?

Suggested answer: Evaluate retrieval against labeled resume passages, tune chunk sizes and top-k values based on those results, and consider hybrid keyword-plus-vector search or a reranker if they measurably improve relevance.

Q37. How could the application scale to more users?

Suggested answer: A production deployment would need explicit session isolation, managed storage and indexing, concurrency controls, rate limiting, authentication, observability, and careful handling of uploaded files and API secrets. These are extensions; the current app is a simple Gradio process.

Q38. What would you do if the model returns malformed sections?

Suggested answer: Keep the current fallback behavior, add tests for common heading variations, and consider enforcing a structured output schema. Validate required fields and show an explicit error or retry rather than presenting malformed content as a successful report.

Q39. What is the most important design trade-off in this project?

Suggested answer: The app combines reusable role guidance with user-specific resume evidence to produce tailored feedback, while keeping the workflow approachable. The trade-off is that results rely on retrieval and a generative model, so the output is advisory and needs verification.

Q40. How should you describe your personal contribution in the viva?

Suggested answer: Describe only the work you personally completed. Be prepared to name the files or components you worked on, explain one implementation decision, and give an example of a challenge and how you addressed it. Do not claim a team contribution as an individual accomplishment.